Efficient Transformer-Based Model for Windows Portable Executable File Malware Detection

Kun Chen
kun.chen@vanderbilt.edu
Vanderbilt University
Nashville, TN, USA

Abstract

With the increasing applications of pervasive computing into daily life, malware continues to cause significant threats to privacy and computer security. As a result, the demand for accurate and efficient malware detection techniques has become more and more necessary. In recent years, Transformer-based models have demonstrated remarkable success across various domains, including computer vision, natural language processing, and tabular data analysis. However, their relatively large computational cost becomes a key limitation. In this paper, we propose an efficient Transformer-based architecture for detecting malware in Windows Portable Executable (PE) files, utilizing the Swarm Optimization and Machine Learning Applied to Portable Executable Malware Detection (SOMLAP) dataset. The proposed model with Layer Normalization, achieves a high classification accuracy of 98.54%. While Layer Normalization is believed as an indispensable component in Transformer architectures, we investigate the potential of *Dynamic Tanh*, a replacement to layer normalization recently introduced by Zhu et al. [22]. Experimental results show that replacing Layer Normalization with Dynamic Tanh results in a slightly lower accuracy of 96.97% and increased training time, suggesting that Dynamic Tanh may be less suitable for tabular data in Transformer-based model for malware detection. Additionally, we explore the effect of the initial value of the learnable parameter α in Dynamic Tanh, and observe that higher initial α values lead to improved model performance. These findings provide valuable insights into the potential of Dynamic Tanh in Transformer based models for security-related tabular data tasks.

Keywords

Security, Privacy, Pervasive Computing, Deep Learning, Malware Detection, Transformer, Layer Normalization, Dynamic Tanh

ACM Reference Format:
Kun Chen. 2025. Efficient Transformer-Based Model for Windows Portable Executable File Malware Detection. In . ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Nowadays, pervasive computing has become a more and more popular concept and has widely applied in people's daily life. Examples of such applications include smart home, wearable devices to monitor health and so on. Even though pervasive computing brings great convenience to people's daily life, security and privacy problem can be a big concern to the application of pervasive computing in people's daily life. Malware is one of the biggest security and privacy problems that bring significant threats to the development of pervasive computing. Continuous health monitoring device has become a more and more popular option for patients as it allowed the patients and hospitals to detect any health concern in the earliest time. However, if malware existed in the devices, the patients' medical privacy might be leaked and stolen. If the hackers changed the data in the monitoring device, it may cause great risk to the patients' life. Another example is smart home. Smart home is a successful application of pervasive computing concept in people's daily life and many people believe it brings great convenience to their life. However, attackers may easily get a lot of privacy data from the smart home by utilizing the malware, causing great concern to the security and privacy. Based on the above two examples, we can see how malware can cause big harm to the security and privacy of the people's daily life and hinder the development and application of the pervasive computing. The development and application of pervasive computing is important, but the security problem must also be paid enough attention to allow users to enjoy the convenience without any worries.

With the rapid development of machine learning (ML) and deep learning (DL), many researchers have applied these techniques to solve computer security issues, including malware detection. Common ML techniques include decision stumps, random trees, stepwise logistic regression, gradient boosting trees, multi-dimensional Naive Bayes, classification and regression trees (CART), LightGBM, and XGBoost [13]. Wadkar et al. [20] proposed malware evolution detection system using support vector machine. Khammas [11] proposed a Ransomware detection system using Random Tree technique and achieved a 97.74% accuracy. Gao [8] proposed a malware detection system using LightGBM With a Custom Logistic Loss Function and achieved a AUC of 0.979 which outperformed the state-of-the-art. DL-based methods such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, Bidirectional LSTM (BiLSTM), and Deep Neural Networks (DNNs) have also been widely applied [13]. Vinayakumar et al. [19] proposed a Android malware detection system based on LSTM and achieved an accuracy of 93.9% on dynamic analysis and an accuracy of 97.5% on static analysis. With the popularity of transformer due to strong power of self-attention

mechanism, more and more transformer-based model are used in solving malware detection.For example, Rahali et al. [15] proposed MalBERT, a Transformer-based model using Bidirectional Encoder Representations from Transformers (BERT) for Android malware detection and achieved a weighted F1-score between 82% and 99% on different datasets.

Malware can appear in a wide range of environments, including Windows Portable Executable (PE) files, Linux, Android, iOS and macOS applications, PDF documents, and HTML/JavaScript files. Malware detection datasets are often built based on specific environments. The dataset used in this study, the Swarm Optimization and Machine Learning Applied to PE Malware Detection (SOMLAP) dataset [10], focuses on malware existed in Windows PE files. Detection tasks typically include two categories: binary classification and multi-class classification. Binary classification is to detect if the malware existed or not. The multi-class classification is to detect if the malware existed or not, and if the malware existed, predict the specific class of malware. The SOMLAP dataset [10] provides binary labels, and thus, this paper focuses on the binary classification task.

Transformer-based models often outperform traditional methods due to their powerful self-attention mechanism, but they also suffer from high computational costs [1]. To address this, Zhu et al. [22] proposed *Dynamic Tanh*, an alternative to Layer Normalization. Their findings show that Dynamic Tanh can reduce training time while maintaining or exceeding performance in various scenarios [22]. However, its effectiveness in Transformer models applied to tabular datasets has not been evaluated. This work is the first study to investigate the use of Dynamic Tanh in Transformer-based models for tabular data and malware detection. We evaluate the performance of this technique based on metrics including accuracy, precision, recall, F1-score and training time.

The main contributions of this paper are summarized as follows:

- Propose an efficient Transformer-based model for Windows PE malware detection using the SOMLAP dataset, achieving a high accuracy of 98.54%.
- Present the first study comparing Dynamic Tanh and Layer Normalization in Transformer-based models applied to tabular data for malware detection. The results show that Dynamic Tanh yields lower accuracy and longer training times in this context.
- Explore the effect of the initial value of the learnable parameter α in Dynamic Tanh and observe that higher initial α values leads to higher accuracy. This finding suggests the necessity of tuning α for optimal performance.

2 Related Work

Malware has causing great damages to the safety and privacy of the society and has become one of the most serious issues of computer security. With the development and popularity of the internet, malware becomes more and more complex and thus harder to detect. Malware can be categorized into 14 distinct classes, including Trojans, Viruses, Worms, Spyware, Rootkits, Botnets, Adware, Ransomware, Keyloggers, Backdoor, Cryptojacking, Fileless malware, and Obfuscated malware [13]. According to [3] and [13], Trojan fakes themselves as benign software and steal the information on the system once executed. Viruses enter the system by attaching themselves to the benign programs and attack the system after entering. Worms can enter computers by replicating themselves. Spyware secretly monitoring the client's activity and steal the information without permission. Rootkit is a software that allows the attackers gain the administrator access to the system without informing or permission. Botnet is a network controlled by the attackers and attackers used Botnet to conduct attacks to the computers within the system. Crimeware is a malware that is used for criminal purposes. Adware is the software that can deliver advertisement without permission from the users. Ransomware attacks and steals clients' privacy and stop until ransom is paid. All these malware bring great damages and issues to people's daily life and thus building efficient malware detection system becomes extremely necessary.

Different approaches have been applied to conduct the malware detection. One of the most popular approaches is machine learning and deep learning techniques. Nowadays, machine learning and deep learning techniques have been widely used in a great number of areas including vision recognition, large language models, speech recognition, translation, auto-driving and so on. According to [13], common machine learning methods used for malware detection include Random Forest, Binary Logistic regression, gradient boosting tree, Naive Bayes, K-means, LightGBM, support vector machine and so on. According to [3], multi-layer perception (MLP), KNN is also popular deep learning techniques used for malware detection. Vaswani et al. [18] proposed transformer and multi-head attention mechanism which soon become popular and being widely used in a large number of areas including vision, large language models, speech recognition, translation and tabular data processing. The multi-head attention mechanism can capture the complex relationship between tokens and thus produce great performance. Successful application of transformer include BERT, GPT, Vision Transformer (ViT). Although Transformers are not traditionally seen as appropriate for tabular data, there have been successful applications, such as TabTransformer [9], demonstrating their potential effectiveness in this domain. Big success of applying transformer and especially the multi-head attention mechanism implies the application of transformer in malware detection is promising. Seneviratne et al. [17] proposed SHERLOCK which is an self-supervised vision transformer based model for malware detection and achieved 97% accuracy for the binary classification based on the 1.2 million Android applications. Devlin et al. [6] proposed Bidirectional Encoder Representations from Transformers which soon become popular and been applied in various domains. Rahali et al. [15] proposed MalBERT, which is a Bidirectional Encoder Representations from Transformers (BERT) based models for malware detection and achieved a promising and great performance based on the source code of Android applications. Deng et al. [5] proposed TransMalDE which is a transformer based framework for malware detection that is specifically designed for the malware detection in IoT networks.

Many applications of transformer for malware detection have been proven to be successful and promising. However, Alshomrani [1] pointed out that transformer-based model suffered from several challenges or limitations compared to other methods. The first one is data requirements which means a large amount of data are needed to make the transformer-based model being trained

effectively. The second one is unbalanced data which means balanced datasets are required to train the transformer-based model effectively. The third limitation is large computational resources. Due to the self-attention mechanism, even though it brings great power to the good performance of the transformer-based model, it also required much larger computational resources compared to other machine learning or deep learning methods. There have been some popular datasets for malware detection including EMBER dataset [2], Microsoft Big 2015 [16], CIC-MalMem-2022 [7]. However, these popular datasets may not be the appropriate choice for training a transformer-based model. For example, Drebin dataset only includes 5560 samples which is relatively low number for training an effective transformer-based model. EMBER dataset contained features from 1.1 million portable executable files, which is large enough to train an effective transformer-based model. However, due to computational resource limit, training such large datasets is difficult. In this paper, we utilized Swarm Optimization and Machine Learning Applied to PE Malware Detection datasets (SOMLAP) proposed by Kattamuri et al. [10]. This dataset contained 51409 samples including both benign and malware files and 108 pure Portable Executable File header attributes which allowed the attention mechanism to capture the complex relationship between the attributes to improve the malware detection. The dataset samples is affordable but also enough for training an efficient transformer-based model.

Zhu et al. [22] proposed a new technique called Dynamic Tanh which can be used as replacement to the layer normalization which challenges the conventional ideas that layer normalization is indispensable in the transformer-based model. The output from layer normalization is calculated as

$$\text{Normalization}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

[22] [4]. However Zhu et al proosed a new formula

$$\text{DyT}(x) = \gamma \cdot \tanh(\alpha x) + \beta$$

where α is a learnable scalar parameter and γ and β are learnable vector parameters [22]. Zhu et al. [22] tested the effectiveness of Dynamic Tanh in a variety of domains including supervised learning in vision, self-supervised learning in vision, diffusion models, large language models, self-supervised learning in speech. The result demonstrates that both the inference time and training time are decreased and might be used as potential choice for efficiency-oriented model architecture design. Even though the training time decreased, the model performance using dynamic tanh can match or even exceed the performance of model using layer normalization. Since one limitation of transformer-based models is their high computational cost, we propose replacing layer normalization with Dynamic Tanh to evaluate its effectiveness. As previous studies have not tested Dynamic Tanh within transformer architectures for tabular data and this is the first work to explore this approach.

3 Problem Statement

(1) Propose a transformer-based model for malware detection on Windows Portable Executable (PE) files and evaluate its performance using metrics including accuracy, recall, precision, F1-score, and training time.
(2) Investigate the effectiveness of Dynamic Tanh in the proposed transformer-based model and compare its performance with the same model with Layer Normalization.
(3) Analyze the impact of different initial values of the α parameter in Dynamic Tanh on the model's performance.

3.1 Proposed Model Architecture

We proposed a Transformer-based architecture 1, named K-transformer, which is used to detect Windows Portable Executable Files Malware Detection. The architecture follows the standard Transformer encoder structure combined with a multi-layer perception as the classification head. Since most of the transformer-based models are computationally expensive compared to other methods, we replaced the layer normalization in the transformer encoder with a new technique called Dynamic Tanh which is introduced by Zhu et al. [22]. According to [22], replacing Layer Normalization with Dynamic Tanh can not only match or exceed the model performance but also lowered the training time, which would be beneficial to a computationally expensive model. However, since the authors did not test the effectiveness of Dynamic Tanh on a transformer-based model used for tabular datasets, this is the first work to test if Dynamic Tanh can work well on a transformer-based model for tabular datasets. The self-attention mechanism in each Transformer Block enables the model to understand the importance of each feature in the dataset and thus learn how different features of the input vectors relate to each other. The multi-layer perception serves as the final classification head which transforming the high-level representation output from the transformer encoder into the prediction probabilities, either normal or malware. The model is divided into three main parts:

(1) Embedding layer: The linear embedding layer projects the input vectors to a vector with embedding dimension to prepare it for the transformer encoder. Since the datasets is in tabular form, reshape the vector into (Batch Size, Sequence Length, Embedding Dimension) which each feature is treated as a token to make it compatible for the transformer encoder architecture.
(2) Transformer Encoder: The transformer encoder in the model basically follows the standard design, which consists multi-head self-attention with dropout so that the model can learn the complex relationships between the tokens and prevent overfitting. The only difference with the standard transformer encoder is that we replaced the layer normalization with the Dynamic Tanh to see if the model can have a better performance and less training time. For the position-wise feedforward network, each transformer encoder block has two fully connected layers and selected GELU as the activation function. There are 8 blocks in the transformer encoder and 16 heads in the multi-head self-attention mechanism. To prevent the overfitting, we set the dropout probability in the multi-head self-attention mechanism as 0.2.
(3) Multi-Layer perception classification head: The output from the transformer encoder will be passed through the multi-layer perception to do the final prediction on if the malware existed. The multi-layer perception classification head includes a linear layer reducing the dimension from 128 to 64,

a ReLU activation function, a dropout with probability of 0.2, and finally a linear layer reducing the dimension from 64 to 2 (normal or malware).

To optimize the performance of the architecture on the dataset, we selected the following configurations, as shown in Table 1.

To test if the Dynamic Tanh can work well in the proposed architecture that is used to process tabular datasets, we also built another model which the architecture is exactly the same as the K-transformer but using the Layer Normalization instead of the Dynamic Tanh. We will compare their performance based on accuracy, precision, recall, F1-score and training time. We also aim to find the impact of the initial α value on the performance of the model, so we build the model which the architecture is exactly the same as the K-transformer but with different initial α value. Due to computational resources limit, we only selected three values: 0.3, 0.5 and 0.7.

Table 1: K-Transformer Model Hyperparameter Configuration

Hyperparameter	Value
Embedding Dimension	128
Number of Encoder Blocks	8
Number of Attention Heads	16
Activation Function (Transformer Encoder)	GELU
Activation Function (MLP Head)	ReLU
Dropout Probability (MHSA)	0.2
Dropout Probability (MLP)	0.2
Optimizer	Adam
Learning Rate	1e-4
Batch Size	128
Loss Function	CrossEntropyLoss()
Initialized α in DyT	0.5

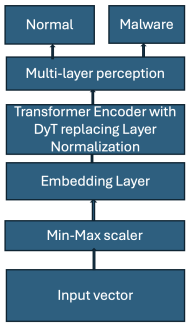

Figure 1: Model Architecture of K-Transformer

4 Evaluation

Setup

For our experiments, we used the Swarm Optimization and Machine Learning Applied to Portable Executable Malware Detection dataset (SOMLAP). This dataset contains a total of 51,409 samples and 108 numerical features extracted from the headers of Windows Portable Executable (PE) files [10]. Among them, 19,809 are malware samples collected from VirusShare, while 31,600 are benign samples obtained from the Windows 10 Operating System.

To ensure reliable evaluation, we split the dataset into three parts: 80% for training, 10% for validation, and 10% for testing. A stratified train-test split was applied to maintain the original class distribution across all subsets, which is essential for balanced performance evaluation.

Since all features in the dataset are numerical, we applied Min-Max scaling to normalize each feature to the range [0,1]. Min-Max scaling preserves the relative relationships among feature values and prevents features with relatively small values from being underrepresented during training. For each feature X, the scaled value X' is computed using:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

where $X_{\max}$ and $X_{\min}$ represents the maximum and minimum values of the feature, respectively. As a result, the minimum value is scaled to 0 and the maximum value is scaled to 1.

We used the Cross-Entropy Loss function, which is well-suited for classification tasks such as malware detection [14]. To optimize the model, we employed the Adam optimizer [12] with a learning rate of 0.0001, which balances convergence speed and training stability.

The model was trained for 20 epochs on Google Colab using an NVIDIA T4 GPU, which significantly accelerated the training process compared to CPU-only environments.

Baselines

Since the Swarm Optimization and Machine Learning Applied to Portable Executable Malware Detection (SOMLAP) dataset [10] is relatively new—published in 2023, there are limited studies conducted based on it. To provide a meaningful evaluation, we selected several widely-used machine learning algorithms that are commonly applied in malware detection tasks as baselines. These include Logistic Regression (LR), Gaussian Naive Bayes (GNB), and Support Vector Machine (SVM). In addition, we included a Multi-Layer Perceptron (MLP) to compare deep learning-based approaches.

To evaluate the effectiveness of the Dynamic Tanh (DyT), we built a K-Transformer architecture using Layer Normalization instead of DyT. This allows for a direct comparison for the effectiveness of Layer Normalization and Dynamic Tanh on the model performance.

Furthermore, to investigate the impact of initial α value in Dynamic Tanh on the model performance, we included additional variants of the K-Transformer with different initial α values: 0.3 and 0.7. These variants help to assess how the choice of initial α value impacts training time and classification performance.

Evaluation Metrics

To comprehensively evaluate the performance of the proposed architecture, we select five evaluation metrics: accuracy, precision, recall, F1-score, and training time.

Accuracy measures the proportion of correctly classified samples among all test samples. It reflects the overall correctness of the

model in the malware detection task. The formula is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is the number of true positives, TN is true negatives, FP is false positives, and FN is false negatives.

Precision measures the proportion of correctly predicted positive samples out of all samples predicted as positive. It evaluates the model's reliability when it predicts a sample as malware:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall is the proportion of correctly predicted positive samples out of all actual positive samples. It reflects the model's ability to detect true malware:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score is calculated based on precision and recall. It provides a balanced evaluation between precision and recall.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Training time is used to assess the computational efficiency of the model. To ensure fairness in comparison, all models including the baselines are trained under the same setting: 20 epochs on Google Colab using a T4 GPU.

Result Analysis

Table 2: Accuracy (%) of different model architectures on the SOMLAP test set

Model	Accuracy (%)
K-Transformer (α=0.5)	96.73
K-Transformer (α=0.3)	87.18
K-Transformer (α=0.7)	96.97
K-Transformer (LayerNorm)	98.54
Multi-Layer Perceptron	94.36
Logistic Regression	90.96
Gaussian Naive Bayes	62.44
Support Vector Machine	96.32

The accuracy of different model architectures on the SOMLAP datasets are shown in Table. 2. Among the eight evaluated model architectures, the K-Transformer with Layer Normalization achieved the highest accuracy of 98.54%. Gaussian Naïve Bayes exhibited the poorest performance, with an accuracy of only 62.44%. The Multi-Layer Perceptron achieved an accuracy of 94.36%. Support Vector Machine achieved an accuracy of 96.32%. Notably, despite its high accuracy, the Support Vector Machine's training time was significantly shorter than the Transformer-based models, suggesting a good trade-off between efficiency and performance.

Within the Transformer-based models using Dynamic Tanh (DyT), the K-Transformer with an initial $\alpha = 0.7$ performed best, achieving an accuracy of 96.97%. The model with $\alpha = 0.5$ achieved an accuracy of 96.73%. The model with $\alpha = 0.3$ only achieved an accuracy of 87.18%. These results indicate that while DyT can maintain competitive performance, it does not outperform Layer Normalization in accuracy on tabular data. However, Layer Normalization is traditionally considered indispensable in Transformer models, and the findings suggest that replacing the layer normalization with DyT can still preserve strong performance, even though there is a slight drop in accuracy. The precision of different model architectures on the SOMLAP datasets are shown in Table. 3. K-Transformer with Layer Normalization achieved the best precision for both classes. The K-transformer using DyT with $\alpha = 0.5$ and $\alpha = 0.7$ also achieved the great precision. Specifically, the model with the initial alpha value of $\alpha = 0.5$ reached a malware precision of 98.66%, only 0.01% below the model using Layer Normalization, indicating high reliability when predicting malware. Notably, Gaussian Naïve Bayes—achieved a malware precision of 98.09% even though it has low overall accuracy, suggesting it is also reliable when it does make a positive prediction.

Table 3: Precision (%) of different models on the SOMLAP test set

Model	Normal	Malware
K-Transformer (α=0.5)	95.64	98.66
K-Transformer (α=0.3)	90.20	82.56
K-Transformer (α=0.7)	97.19	95.70
K-Transformer (LayerNorm)	98.46	98.67
Multi-Layer Perceptron	95.35	92.77
Logistic Regression	95.02	85.29
Gaussian Naive Bayes	62.08	98.09
Support Vector Machine	95.28	98.17

Table 4: Recall (%) of different models on the SOMLAP test set

Model	Normal	Malware
K-Transformer (α=0.5)	99.21	92.78
K-Transformer (α=0.3)	88.80	84.60
K-Transformer (α=0.7)	97.31	95.51
K-Transformer (LayerNorm)	99.18	97.53
Multi-Layer Perceptron	95.47	92.58
Logistic Regression	90.00	92.48
Gaussian Naive Bayes	99.97	2.57
Support Vector Machine	98.92	92.18

The recall of different model architectures on the SOMLAP datasets are shown in Table. 4. Gaussian Naïve Bayes achieved the highest recall for the normal class at 99.97%, while the K-Transformer with Layer Normalization achieved the highest recall for malware at 97.53%. This implies that Gaussian Naïve Bayes is the most effective at detecting actual benign samples, and the K-Transformer with LayerNorm is the most effective at detecting actual malware samples.

The F1-score of different model architectures on the SOMLAP datasets are shown in Table. 5. K-Transformer with Layer Normalization again achieved the best results for both classes, reflecting a strong balance between precision and recall. The DyT-based models with $\alpha = 0.5$ and $\alpha = 0.7$ also performed well, just behind the

Table 5: F1-score (%) of different models on the SOMLAP test set

Model	Normal	Malware
K-Transformer (α=0.5)	97.39	95.63
K-Transformer (α=0.3)	89.49	83.57
K-Transformer (α=0.7)	97.25	95.60
K-Transformer (LayerNorm)	98.82	98.10
Multi-Layer Perceptron	95.41	92.67
Logistic Regression	92.44	88.74
Gaussian Naive Bayes	76.59	5.02
Support Vector Machine	97.07	95.08

LayerNorm variant. In contrast, although Gaussian Naïve Bayes showed high precision and recall on individual class, the F1-score for malware was only 5.02%, due to the significant imbalance between precision and recall, making it the lowest among all models evaluated.

Table 6: Training time (in seconds) of different model architectures

Model	Training Time (s)
K-Transformer (α=0.5)	131.36
K-Transformer (α=0.3)	126.73
K-Transformer (α=0.7)	127.18
K-Transformer (LayerNorm)	121.23
Multi-Layer Perceptron	10.36
Logistic Regression	0.32
Gaussian Naive Bayes	0.04
Support Vector Machine	33.32

The training time of different model architectures on the SOMLAP datasets are shown in Table. 6. While Transformer-based models outperform classical methods in accuracy, the training time is significantly longer than other models. Among the four Transformer-based models, the model with Layer Normalization had the shortest training time at 121.23 seconds. Among models using DyT, the one with the initial α value of 0.3 had training time of 126.73 seconds, even though it achieved the lowest accuracy among the three. The model with initial α value of 0.7 required slightly longer time 127.18 seconds but achieved significantly better performance. The model with initial α value of 0.7 had the longest training time, which is 131.36 seconds.

5 Limitations

This study presents several limitations that suggest futre research directions.

First, due to computational resources and time limitation, we were only able to evaluate a limited set of initial α values for the model using Dynamic Tanh. This limits the analysis on how the initial α value impacts model performance. Future work will explore a broader range of initial α values, including 0.1, 0.2, 0.4, 0.8, and others, to better understand the impact of initial α values on the model performance.

Second, although a number of Transformer-based architectures have been proposed for malware detection in recent years, we were unable to include these state-of-the-art models in our baseline models to have comparisons due to resource limitations. In future work, we plan to implement and evaluate these architectures on the SOMLAP dataset to have a more comprehensive comparison against our proposed K-Transformer architecture.

Third, our experiments were conducted solely on the SOMLAP dataset. While SOMLAP is a recently published and well-structured dataset for PE malware detection, the findings based on one single dataset can not represent the general situation. In future studies, we intend to train the porposed the model and investigate the impact of initial α values on model performance with more malware detection datasets in tabular form. This will allow us to more comprehensively evaluate the performance of the proposed architecture and the effectiveness of Dynamic Tanh across diverse datasets in different environments.

6 Discussion

6.1 Dynamic Tanh in Transformer-based model for tabular data

According to the paper, replacing layer normalization with dynamic Tanh can match or exceed the model performance but also decrease the training time in most of the situations. The authors did not test the performance of Dynamic Tanh in a transformer-based model for tabular datasets. This is the first work to test the effectiveness of dynamic tanh in a transformer-based model for tabular datasets. Dynamic Tanh in the proposed architectures did not work bad, however, it also did not match or exceed the model using layer normalization, either in accuracy or training time. We can see with different initial α values, the model's accuracy and training time can be different, demonstrating the impact of initialized alpha value on the model performance. For the accuracy, we can see that with higher initial α values, the higher accuracy achieved. When the initial α values is 0.3, the accuracy is only 87.18%. When the initial α values increased to 0.5, the accuracy increased to 96.73%. When the initial α values is 0.7, the accuracy increased to 96.97%. We can see when the initialized alpha value increased from 0.3 to 0.5, there is a nearly 10% increase in the accuracy. When the initialized alpha value increased from 0.5 to 0.7, the increase in accuracy is only 0.24%, which demonstrates the findings from Zhu et al. [22] that even the default initial α values of 0.5 can not reach a optimal performance, it can achieve a near-optimal performance. However, due to the resource limit, we are not able to perform a more comprehensive evaluation on how the initial α values impact the model performance. In this paper, we only choose the initial α values of 0.3, 0.5 and 0.7. More value like 0.1,0.2,0.4,0.6,0.8 should be chosen to have a more comprehensive evaluation, which can be done in the future work. Even though Dynamic Tanh did not perform well in the proposed model architecture on SOMLAP dataset, it did not necessarily mean it can not perform well in other Transformer-based model for tabular datasets. More experiments regarding how the Dynamic Tanh work in transformer-based model are needed to have a more comprehensive understanding. Even though in this paper, Dynamic Tanh did not work as expected, it is still very

promising since the conventional idea believes that layer normalization is necessary in transformer-based model, but replacing layer normalization with dynamic tanh did not cause a big decrease in the accuracy but only around 1.6% accuracy, which is acceptable. This means that how exactly each part in neural network work needs to be further investigated to have a better understanding of their role, thus providing insights for the future improvement of the existing neural network.

6.2 Comparison between eight model architectures

In this paper, we built and trained a total of eight models, including K-transformer with different initial α values, K-transformer using layer normalization, Multi-layer Perception, Logistic Regression, Gaussian Naive Bayes, Support Vector Machine. All of these models are popular machine learning or deep learning techniques. From result analysis, we can see that K-transformer with layer normalization achieves the highest accuracy, 98.54%. However, we should also notice that the training time is also the one of the longest 121.23 seconds. Other transformer-based model also has very long training time. K-transformer with initial α value of 0.5 has a training time of 131.36 seconds. K-transformer with initial α value of 0.3 has the training time of 126.73 seconds. K-transformer with initial α value of 0.7 has training time of 127.18 seconds. However, for other model architectures, multi-layer perception only trained for 10.36 seconds. Logistic regression only needs 0.32 seconds. Gaussian Naive Bayes only needs 0.04 seconds. Support vector machine only needs 33.32 seconds. All these non-transformer-based model requires far less computational resources compared to transformer-based model. But at the same time, these non-transformer-based model also achieved the great performance. For example, multi-layer perception achieved a 94.36% accuracy. Logistic regression achieved a 90.96%accuracy. Support vector machine achieved a 96.32% accuracy, which is even much higher than the K-transformer with initial α value of 0.3 but requires far less training time and far less computational resources. Nowadays, with the quick development and popularity of deep learning and artificial intelligence, more and more companies and researchers joined the artificial intelligence group to train models to solve problems in different area including some area which previously thought has no relationship with computer science or artificial intelligence like biology or chemistry. The popularity of AI caused an huge increase in the needs for computational resources, especially for GPU, making GPU more and more expensive. Companies or researchers have to balance the advantages and disadvantages of different models. Transformer-based model suffered from many limitations as mentioned before including high computational resources, requiring high quality and quantity datasets. Even though sometimes transformer-based model can bring surprising performance thanks to the self-attention mechanism, the transformer-based model may not always work well especially when the computational resources are limited or the datasets quality and quantity is not high. In this case, choosing other models or methods are more realistic. We can see that using some simple machine learning methods like support vector machine or simple deep learning models like multi-layer perception can also achieved a pretty good performance but required far less computational resources and training time. If the researchers had enough computational resource and had a high quality datasets with enough quantity, transformer-based model can be a ideal option to achieve a better performance than the state-of-the-art. Otherwise, consider some basic machine learning techniques or deep learning techniques like random forest, support vector machine, multi-layer perception sometimes can also get a surprising performance.

6.3 Further improvement to the K-transformer

This paper's main focus is on how Dynamic Tanh can impact the model performance in either accuracy and training time and the comparison between Dynamic Tanh and Layer Normalization in the transformer-based model. Even though the K-transformer with layer normalization already achieved a very high accuracy on the SOMLAP dataset, there are still further improvement can be investigated to improve the K-transformer. In the model architecture, we select the multi-layer perception as the final classification head, which is a classic choice as the lot of other transformer-based model like Vision Transformer or Tab-Transformer do. However, there are other classification head like Cosine Similarity classification head can be used to see if an improvement can be achieved. Besides classification head, other model can be combined with the transformer. Zhang et al. [21] proposed Transformer-LSTM model which combined Transformer and LSTM before the classification, and achieved a 99.00% accuracy on NSL - KDD dataset. For the future improvement to the K-transformer, we can choose to add LSTM or other neural network architectures to the model and also see how the impact of Dynamic Tanh will change with the different model architectures. In this paper, we focused on malware detection. However, this model architectures can also apply to other areas including network intrusion, phishing detection, botnet detection. There are also many other classic datasets used for malware detection including Drebin datasets or EMBER dataset, we can train the proposed model architecture on these dataset and compare with the state-of-the-art. Due to resource limit, the hyperparameter tuning on the validation datasets is really limited, which means that the model's performance might not be optimized. Further improvement on the K-transformer can be also focused on hyperparameter tuning to see if a better performance can be achieved.

6.4 Future directions

There are several future directions can be done regarding this paper. The first future direction is to further investigate the impact of dynamic tanh on the transformer-based model for tabular dataset. There are a lot of stat-of-the-art work on transformer-based model for tabular dataset including Tab-Transformer and FT-Transformer. We can replace the layer normalization with dynamic tanh in these model architectures and see how dynamic tanh work on these model architectures to see which one works better. Besides transformer and transformer-based model, some other neural network also utilized layer normalization such as recurrent neural network or multi-layer perception in some cases. We can also replace the layer normalization with dynamic tanh in these non-transformer-based model to see if an improvement can be made, either in model

performance or computational resources. Besides, we can also replace the SOMLAP dataset with other tabular datasets to see if dynamic tanh can work better than layer normalization.

The second future direction is add more metrics regarding the computational resource. In this paper, we calculated the training time as the metrics of the computational resource since all the transformer-based models and multi-layer perception are trained with 20 epochs and all the models are trained under T4 GPU. However, there are other metrics can be used to measure the computational cost including inference time, GPU memory usage, energy usage. In the future work, we should add these metrics to have a more comprehensive evaluation on the model's computational resources consumption.

The third future direction is to implement the model architectures for different area of computer security. Even though this model is originally designed for windows portable executable file malware detection and trained with SOMLAP dataset, this model architecture can be used to be trained with any kinds of datasets in tabular form and thus this model architecture can be used to do other kinds of tasks. For example, we can train this model architecture with NSL - KDD datasets to do the network intrusion task. We can also train this model architecture with datasets designed for Android malware detection. We can also train this model with datasets designed for specific types of malware including Ransomware, Spyware, Backdoor, Adware, Virus, Botnet and so on. Overall, this model architecture can be used for most of the datasets in tabular form with just small modifications required and great potential of this model architecture and dynamic tanh can be further investigated.

The fourth future work direction is how the initial α value affect the model performance. Due to the resource limit, we only select three initial α value, which is the default initial α value 0.5, 0.3 and 0.7. Based on the experiments, we found out that with higher initial α value, the higher accuracy can be achieved. This means the model performance is likely to be further optimized if we choose an initial alpha value larger than 0.7, like 0.8, 0.9 or even 1. The impact of initial α value on different model architectures can also likely to be different. We can also consider implementing the dynamic tanh with different initial α value in other transformer-based model for tabular datasets like TabTrasnformer or FT-transformer. The impact of initial α value on the model performance may also vary depends on the datasets. We can also trained the models with different datasets to see if the impact of initial α value changed. Overall, there is a great potential on how the impact of initial α value changes the model performance, which can provide valuable insights on how the dynamic tanh or layer normalization works in the transformer-based model.

7 Conclusion

In conclusion, this paper proposed an efficient transformer-based model used for Windows portable executable files malware detection based on SOMLAP dataset, which achieved a 98.54% accuracy. This paper is the first work to investigate the potential of replacing layer normalization with dynamic tanh in a transformer-based model for tabular data. The experiments shows that the model with layer normalization outperforms the same model architecture with dynamic tanh and with less training time. But with the appropriate initial α value, the proposed model can also achieve a good performance, only around 1.5% less accuracy than the proposed model with layer normalization. This paper also demonstrates the impact of initial α value on the model performance. In the proposed model architecture, with higher initial alpha value, the higher accuracy it can achieved.

References

[1] Mohammed Alshomrani, Aiiad Albeshri, Badraddin Alturki, Fouad Shoie Alallah, and Abdulaziz A. Alsulami. 2024. Survey of Transformer-Based Malicious Software Detection Systems. *Electronics* 13, 23 (2024). doi:10.3390/electronics13234677

[2] Hyrum S. Anderson and Phil Roth. 2018. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. arXiv:1804.04637 [cs.CR] https://arxiv.org/abs/1804.04637

[3] Muhammad Azeem, Danish Khan, Saman Iftikhar, Shaikhan Bawazeer, and Mohammed Alzahrani. 2024. Analyzing and comparing the effectiveness of malware detection: A study of machine learning approaches. *Heliyon* 10, 1 (2024), e23574. doi:10.1016/j.heliyon.2023.e23574

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. arXiv:1607.06450 [stat.ML] https://arxiv.org/abs/1607.06450

[5] Xiaoheng Deng, Zhe Wang, Xinjun Pei, and Kaiping Xue. 2024. TransMalDE: An Effective Transformer Based Hierarchical Framework for IoT Malware Detection. *IEEE Transactions on Network Science and Engineering* 11, 1 (2024), 140–151. doi:10.1109/TNSE.2023.3292855

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[7] Canadian Institute for Cybersecurity. [n. d.]. Malware memory analysis (CIC-MalMem-2022). https://www.unb.ca/cic/datasets/malmem-2022.html/. Accessed: 2025-04-13.

[8] Yun Gao, Hirokazu Hasegawa, Yukiko Yamaguchi, and Hajime Shimada. 2022. Malware Detection Using LightGBM With a Custom Logistic Loss Function. *IEEE Access* 10 (2022), 47792–47804. doi:10.1109/ACCESS.2022.3171912

[9] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. arXiv:2012.06678 [cs.LG] https://arxiv.org/abs/2012.06678

[10] Santosh Jhansi Kattamuri, Ravi Kiran Varma Penmatsa, Sujata Chakravarty, and Venkata Sai Pavan Madabathula. 2023. Swarm Optimization and Machine Learning Applied to PE Malware Detection towards Cyber Threat Intelligence. *Electronics* 12, 2 (2023). doi:10.3390/electronics12020342

[11] Ban Mohammed Khammas. 2020. Ransomware Detection using Random Forest Technique. *ICT Express* 6, 4 (2020), 325–331. doi:10.1016/j.icte.2020.11.001

[12] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG] https://arxiv.org/abs/1412.6980

[13] Pascal Maniriho, Abdun Naser Mahmood, and Mohammad Jabed Morshed Chowdhury. 2024. A systematic literature review on Windows malware detection: Techniques, research issues, and future directions. *Journal of Systems and Software* 209 (2024), 111921. doi:10.1016/j.jss.2023.111921

[14] Anqi Mao, Mehryar Mohri, and Yutao Zhong. 2023. Cross-Entropy Loss Functions: Theoretical Analysis and Applications. arXiv:2304.07288 [cs.LG] https://arxiv.org/abs/2304.07288

[15] Abir Rahali and Moulay A. Akhloufi. 2021. MalBERT: Malware Detection using Bidirectional Encoder Representations from Transformers. In *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 3226–3231. doi:10.1109/SMC52423.2021.9659287

[16] Royi Ronen, Marian Radu, Corina Feuerstein, Elad Yom-Tov, and Mansour Ahmadi. 2018. Microsoft Malware Classification Challenge. arXiv:1802.10135 [cs.CR] https://arxiv.org/abs/1802.10135

[17] Sachith Seneviratne, Ridwan Shariffdeen, Sanka Rasnayaka, and Nuran Kasthuriarachchi. 2022. Self-Supervised Vision Transformers for Malware Detection. *IEEE Access* 10 (2022), 103121–103135. doi:10.1109/access.2022.3206445

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[19] R. Vinayakumar, K.P. Soman, Prabaharan Poornachandran, and S. Sachin Kumar. 2018. Detecting Android malware using Long Short-term Memory (LSTM). *Journal of Intelligent & Fuzzy Systems* 34, 3 (2018), 1277–1288. doi:10.3233/JIFS-169424 arXiv:https://journals.sagepub.com/doi/pdf/10.3233/JIFS-169424

[20] Mayuri Wadkar, Fabio Di Troia, and Mark Stamp. 2020. Detecting malware evolution using support vector machines. *Expert Systems with Applications* 143 (2020), 113022. doi:10.1016/j.eswa.2019.113022

[21] Zhipeng Zhang, Xiaotian Si, Linghui Li, Yali Gao, Xiaoyong Li, Jie Yuan, and Guoqiang Xing. 2023. An Intrusion Detection Method Based on Transformer-LSTM Model. In *2023 3rd International Conference on Neural Networks, Information and Communication Engineering (NNICE)*. 352–355. doi:10.1109/NNICE58320.2023.10105733

[22] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. 2025. Transformers without Normalization. arXiv:2503.10622 [cs.LG] https://arxiv.org/abs/2503.10622